Implementation and Ramifications of TAMER-OP: a Toolkit for Algebraic Module Encodings over $\mathbb{R}^n$ and Other Posets

Erik Mendes Novak

March 17, 2026

Contents

1 Introduction **2**

2 Multiparameter persistence and its difficulties **3**
2.1 Posets, filtrations, and persistence modules . . . 3
2.2 The one-parameter scope . . . 4
2.3 What changes in several parameters . . . 4
2.4 What one computes in place of a barcode . . . 5

3 Tameness, finite encodings, and computability **6**
3.1 Finite encodings . . . 6
3.2 Tame modules and tame morphisms . . . 7
3.3 Upsets, downsets, and fringe presentations . . . 7
3.4 Why encoding is the right answer . . . 8

4 Overview of TAMER-OP as an encoding-centered system **9**

5 Pre-encoding objects and data interfaces **10**
5.1 Finite-poset presentations: finite fringe presentations and indicator data . . . 10
5.2 $\mathbb{Z}^n$ front end: flange presentations and finitely determined data . . . 11
5.3 $\mathbb{R}^n$ front-end: PL fringes and box presentations . . . 12
5.4 Raw data, filtrations, and labeled cell complexes . . . 13

6 The encoding step **14**
6.1 Statement of the encoding problem . . . 14
6.2 Signatures, constant regions, and finite model posets . . . 14
6.3 The finite-poset case . . . 16
6.4 Encoding flange presentations over $\mathbb{Z}^n$ . . . 16
6.5 Encoding PL and box presentations over $\mathbb{R}^n$ . . . 18
6.6 Geometry attached to an encoding map . . . 22
6.7 From filtered data to finite encodings . . . 22
6.8 Common encodings, compiled evaluation, and reusable results . . . 23
6.9 Complexity, diagnostics, and the role of the encoding chapter . . . 24

7 Bibliography notes **24**

1 Introduction

Persistent homology is most useful when it converts filtered geometric data into algebraic objects that are simultaneously expressive and computable. In the one-parameter setting these two sides align particularly well. A filtration indexed by a total order gives rise to a persistence module whose structure is rigid enough to admit interval-type decompositions, barcode summaries, and stable diagrammatic invariants [ELZ02, CEH07, CB15, EH10]. In several parameters, however, this balance breaks down. The indexing poset is no longer a chain, a barcode classification is no longer possible in general, and the cost of direct algebraic manipulation grows quickly with the size and geometry of the parameter space [CZ09, Les15, CDSGO16, Oud15]. Multiparameter persistence therefore forces a design choice: one can either work primarily with summary invariants adapted to specific tasks, or one can try to preserve a richer internal algebraic model on which broader constructions remain possible.

The software library TAMER-OP is specifically concerned with the second viewpoint. Its central principle is that the decisive computational move is not a particular invariant, nor a particular data structure for one family of modules, but rather the explicit construction of a finite encoding. Starting from raw data, finite-poset presentations, flange presentations over $\mathbb{Z}^n$, or piecewise-linear presentations over $\mathbb{R}^n$, TAMER-OP first compresses the ambient parameter poset into a finite encoding poset P together with a classifier or encoding map π. The original object is then represented by a finite-poset module, together with pushed-down presentation data on P. Once this finite internal model has been built, the rest of the library's toolkit becomes uniformly available: module morphisms, kernels and cokernels, indicator resolutions, derived functors, rank- and slice-based invariants, serialization, featurization, and visualization can all be performed against the same encoded object. In this sense, encoding is not merely preprocessing, but indeed is the organizing principle of the entire system.

This viewpoint is directly motivated by Miller's theory of tame modules over posets [Mil20]. Miller's framework shows that, under appropriate finiteness hypotheses, several apparently different descriptions of a module are equivalent: finite encodings, indicator presentations by upsets and downsets, and indicator resolutions all express the same controlled form of tameness. That equivalence extends from the theoretical to a strong implementation consequence. Rather than building separate pipelines for geometric data, algebraic summaries, and downstream statistics, one can work to make the encoding step explicit and robust, and then treat the encoded module as the shared internal object from which all later computations are derived.

TAMER-OP consequently accordingly advances three intertwined claims. Firstly, the finite-encoding perspective provides a mathematically coherent bridge between large ambient parameter spaces and finite combinatorial models. Secondly, this bridge is sufficiently explicit to support a unified software architecture that begins with multiple front end representations and data-ingestion pipelines, and ends with exact algebraic, invariant, and statistical workflows. Thirdly, this architecture is practical, as it supports tractable computation on data sets of ordinary scientific scale and is computationally competitive against existing multiparameter persistence libraries where there is implementation overlap.

We are able to accomplish much through TAMER-OP in part due to our language choice. The library is written in `Julia`, a performant yet high-level programming language, allowing tractability while maintaining clarity and numerical expressiveness.

In the remainder of this thesis, we first introduce the mathematical machinery behind multiparameter persistence and tame modules Then, we overview TAMER-OP, from the front end input, to the encoding step, to downstream computations serialization and visualization. Finally, we benchmark the library to demonstrate its tractability. We hope to demonstrate that nearly every distinctive feature of the library is best understood as more than just an isolated

implementation trick, but as a consequence of making the finite encoding step explicit and central.

2 Multiparameter persistence and its difficulties

Persistent homology begins with a filtered topological object and produces algebra. One chooses an indexing poset, usually a subset of $\mathbb{R}^d$ or $\mathbb{Z}^d$ with the coordinatewise order, and records a vector space at each parameter value together with linear maps along the order relation. In one parameter, this construction leads to a theory with a very strong structure theorem. In several parameters, the same definition still makes sense, but the structure theory changes completely. The purpose of this section is to introduce the basic language carefully and to explain why the multiparameter case resists the sort of complete description that is available in one parameter. Throughout, let $\Bbbk$ be a field.

2.1 Posets, filtrations, and persistence modules

The correct indexing objects in persistence theory are partially ordered sets. A poset may be viewed as a category, and a persistence module is then just a functor out of that category.

Definition 1 (Poset)**.** A *partially ordered set* is a set Q together with a relation $\leq$ that is reflexive, antisymmetric, and transitive.

Definition 2 (A poset as a category)**.** Given a poset $(Q, \leq)$, one forms a small category, again denoted Q, whose objects are the elements of Q, and such that there is a unique morphism $q \to q'$ if and only if $q \leq q'$. Composition is forced by transitivity and identity morphisms are forced by reflexivity.

A category of this form is thin: between any two objects there is at most one morphism. This observation is what allows order theory and homological algebra to interact in persistence.

Definition 3 (Q-module, or persistence module)**.** Let Q be a poset and let $\mathbf{Vect}_{\Bbbk}$ denote the category of vector spaces over $\Bbbk$. A *Q-module* is a covariant functor

$$M : Q \longrightarrow \mathbf{Vect}_{\Bbbk}.$$

Equivalently, one specifies a vector space $M(q)$ for each $q \in Q$ and, for each comparable pair $q \leq q'$, a linear map

$$M(q \leq q') : M(q) \to M(q')$$

subject to the functoriality relations

$$M(q \leq q) = \mathrm{id}_{M(q)}, \qquad M(q \leq q'') = M(q' \leq q'') \circ M(q \leq q')$$

whenever $q \leq q' \leq q''$.

A morphism $f : M \to N$ of Q-modules is a natural transformation. Concretely, this means that for each $q \in Q$ there is a linear map $f_q : M(q) \to N(q)$ and, for each $q \leq q'$, the square

$$\begin{tikzcd} M(q) \arrow[r, "M(q\leq q')"] \arrow[d, "f_q"'] & M(q') \arrow[d, "f_{q'}"] \\ N(q) \arrow[r, "N(q\leq q')"] & N(q') \end{tikzcd}$$

commutes.

The persistent homology modules arising from filtered topology fit into this framework in the most direct possible way: if $\mathcal{F} : Q \to \mathbf{Top}$ is a filtration, then for each homological degree i one obtains a Q-module

$$H_i(\mathcal{F}; \Bbbk) : Q \longrightarrow \mathbf{Vect}_{\Bbbk}$$

by functoriality of homology.

The category of Q-modules inherits the basic exact structure of $\mathbf{Vect}_{\Bbbk}$.

Proposition 4 (Pointwise abelian structure)**.** *For any small category $\mathcal{C}$, the functor category $\mathrm{Fun}(\mathcal{C}, \mathbf{Vect}_{\Bbbk})$ is abelian. In particular, when $\mathcal{C} = Q$ is a poset, kernels, cokernels, images, and coimages are computed pointwise [Mac98, Wei94].*

This pointwise abelian structure is the key to computational feasibility. If the indexing poset is finite, then a persistence module is represented by finitely many vector spaces and finitely many structure maps, so abelian constructions reduce to finite linear algebra. If the indexing poset is infinite, the same proposition still holds formally, but it does not provide a usable algorithm. This gap between formal exactness and finite computability is the problem that motivates the encoding viewpoint later in the thesis.

2.2 The one-parameter scope

The one-parameter theory is the model initial case. Let Q be $\mathbb{R}$ or $\mathbb{Z}$ with the usual total order. Then persistent homology produces modules indexed by a chain, and that special geometry forces a rigid structure theory.

Definition 5 (Interval module)**.** Let $I \subseteq \mathbb{R}$ be an interval. The associated *interval module* $\Bbbk_I$ is defined by

$$\Bbbk_I(t) = \begin{cases} \Bbbk, & t \in I, \\ 0, & t \notin I, \end{cases}$$

with structure maps equal to the identity whenever both source and target lie in I, and equal to the zero map otherwise.

Theorem 6 (Barcode decomposition)**.** *Under the usual pointwise-finite and local-finiteness hypotheses from persistence theory, a one-parameter persistence module decomposes as a direct sum of interval modules, uniquely up to isomorphism and reordering of summands [CB15, Oud15].*

This theorem explains why barcodes are so effective. In the one-parameter setting they are more than just heuristic pictures, indeed encoding the module itself under the standard finiteness assumptions. Stability theory then shows that the associated persistence diagrams vary continuously under perturbation of the filtered data [ELZ02, CEH07, CDSGO16]. Thus one-parameter persistence enjoys both classification and stability.

That combination does not generalize. It is natural to ask for the same story in several parameters, but already the algebra of the indexing category becomes too rich.

2.3 What changes in several parameters

For $d \geq 2$, the standard indexing posets are $\mathbb{Z}^d$ or $\mathbb{R}^d$ with the coordinatewise order

$$a \leq b \quad \Longleftrightarrow \quad a_i \leq b_i \text{ for all } i.$$

This change from a chain to a partial order has major consequences. For example, there are now many incomparable pairs. A module is no longer controlled by a single linear progression of births and deaths. Additionally, if one works over $\mathbb{Z}^d$ or $\mathbb{N}^d$, the algebra is equivalent to the algebra of multigraded modules over a polynomial ring.

Proposition 7 (Algebraic model over $\mathbb{N}^d$)**.** *A persistence module indexed by $\mathbb{N}^d$ is equivalent to an $\mathbb{N}^d$-graded module over the polynomial ring $\Bbbk[x_1, \ldots, x_d]$, where multiplication by x_i records the structure map in the ith coordinate direction [CZ09, Oud15].*

This equivalence is powerful, because it brings in multigraded commutative algebra. It is also, however, the source of the difficulty. The polynomial ring in several variables is not a principal ideal domain, multigraded modules need not decompose into interval modules, and indecomposables occur in families rather than as a discrete list. In particular, the multiparameter theory does not admit a barcode theorem comparable to the one-parameter case [CZ09, Les15].

So the obstruction is not just that computations become larger. The problem is also structural: in one parameter, the module itself can be packaged into a complete discrete invariant; but, in several parameters, there is in general no analogous finite complete invariant of that simple form.

This has two immediate consequences for computation.

(i) One should not expect a universal normal form that is both complete and as easy to manipulate as a barcode.

(ii) If one wants to preserve the full algebraic object, rather than only a summary of it, then one needs a different notion of finiteness.

That second point is the central issue addressed by TAMER-OP. The relevant question is not how to force the multiparameter theory back into one-parameter language, but how to represent the module by finite, computable data without dismissing the algebra one later wants to use.

2.4 What one computes in place of a barcode

Although there is no complete barcode-like invariant in general, several families of invariants remain useful. They are partial by necessity, each only recording one aspect of the module rather than all of it. We now overview some main ones.

Definition 8 (Rank invariant)**.** Let M be a Q-module. For each comparable pair $a \leq b$ in Q, the *rank invariant* is

$$\rho_M(a, b) = \operatorname{rank}(M(a \leq b)).$$

The rank invariant records how much information survives from one parameter value to another. In one parameter it determines the barcode. In several parameters it remains important, but is no longer a complete characterization (different modules may have the same rank invariant).

A second common idea is to probe a multiparameter module along one-dimensional subposets.

Definition 9 (Restriction to a chain or slice)**.** If $\lambda : Q' \to Q$ is a monotone map, then the pullback

$$\lambda^* M := M \circ \lambda$$

is a Q'-module. When Q' is totally ordered, $\lambda^* M$ is an ordinary one-parameter persistence module. Its barcode is then called a *slice barcode* or *fibered barcode*, depending on the context.

This recovers the successful one-parameter theory on each slice, and it is the basis of tools such as RIVET [LW15, RIVET]. But again, different multiparameter modules can have the same family of one-dimensional restrictions.

There are also dimension-based invariants. The most basic one is the restricted Hilbert function

$$q \longmapsto \dim_{\Bbbk} M(q).$$

This records the size of each fiber but not the structure maps between fibers.

A related family comes from chain complexes rather than from a single module. If $C^\bullet$ is a chain or cochain complex of modules on a finite encoding poset, one may define the Euler characteristic function

$$\chi_C(q) = \sum_t (-1)^t \dim_{\Bbbk} C^t(q).$$

When $C^\bullet$ arises from filtered topology, this agrees with the alternating sum of the homology dimensions at q. In this thesis, when we speak of *Euler invariants*, we mean invariants built from this Euler characteristic function, together with the finite-grid signed measures obtained from it by Mobius inversion. These invariants are useful because they are cheap to evaluate, easy to visualize, and often effective in statistical pipelines, even though they are far from complete.

The essential point is that none of these constructions replaces the module itself. Rank invariants, slice barcodes, Hilbert functions, and Euler characteristic functions are all informative, but each forgets some part of the structure. This is why the central focus of TAMER-OP is not the design of a single invariant, but rather the construction of a finite algebraic model from which many different invariants and constructions can be extracted.

3 Tameness, finite encodings, and computability

The correct response to the loss of barcode classification is to identify a class of modules that still admit finite descriptions and remain amenable to exact algebraic computation. In this thesis that role is played by tame modules in the sense of Miller [Mil20]. We now explore what finite encodings are and why tameness is so important.

3.1 Finite encodings

The basic finiteness notion is that a module factors through a finite poset.

Definition 10 (Finite encoding)**.** Let Q be a poset and let M be a Q-module. A *finite encoding* of M consists of

(i) a finite poset P,

(ii) a monotone map $\pi : Q \to P$, and

(iii) a P-module H with finite-dimensional stalks,

such that

$$M \cong \pi^* H = H \circ \pi.$$

Here π^* denotes pullback along the poset morphism π.

Definition 11 (Pullback along a poset morphism)**.** If $\pi : Q \to P$ is monotone and H is a P-module, the *pullback* of H along π is the Q-module

$$\pi^* H := H \circ \pi.$$

Thus

$$(\pi^* H)(q) = H(\pi(q))$$

for each $q \in Q$, and the structure map from q to q' is the structure map of H from $\pi(q)$ to $\pi(q')$.

Proposition 12 (Exactness of pullback)**.** *For any monotone map $\pi : Q \to P$, the pullback functor*

$$\pi^* : \mathbf{Vect}_{\Bbbk}^P \longrightarrow \mathbf{Vect}_{\Bbbk}^Q$$

is exact.

Proof. Kernels, cokernels, and finite direct sums in a functor category are computed pointwise. Pullback by precomposition simply evaluates the P-module at $\pi(q)$ instead of at q, so exactness is preserved at every stalk. □

A finite encoding is therefore not a lossy statistic. It is an isomorphic re-expression of the module through a finite index category. If $M \cong \pi^* H$, then the algebraic behavior of M is constant on each fiber of π, and every fiber and structure map of M is recovered from the finite-poset module H together with the map π.

3.2 Tame modules and tame morphisms

With the notion of finite encoding in hand, tameness is simple to state.

Definition 13 (Tame module)**.** Let Q be a poset. A Q-module M is *tame* if it admits a finite encoding.

This definition is stronger than merely asking for finite-dimensional fibers or finite-rank structure maps. It requires the entire module to be controlled by finitely many regions of parameter space. That extra strength is exactly what makes it suitable for computation.

There is, however, a subtle categorical point. If one allows all natural transformations between tame modules, kernels and cokernels need not remain finitely encoded. The correct category is obtained by restricting to morphisms that are themselves compatible with a common finite description.

Definition 14 (Tame morphism)**.** Let M and N be tame Q-modules. A morphism $f : M \to N$ is called *tame* if there exists a common finite constant subdivision, or equivalently a common finite encoding context, in which the component maps of f are constant on each region in the obvious sense [Mil20].

Theorem 15 (Miller)**.** *With tame modules as objects and tame morphisms as morphisms, the resulting category is abelian [Mil20].*

This addresses a natural concern. One might worry that finite encodability is too restrictive to support genuine homological algebra. The point of Miller's framework is that this is not the case: the correct category of tame modules is still abelian, so kernels, cokernels, exact sequences, and derived constructions remain available. Recent work of Waas further clarifies that categories of finitely encodable persistence modules have better exact behavior than one might first expect, reinforcing the view that finite encodability is a mathematically natural rather than ad hoc hypothesis [Waa24].

Thus, the word tame should not be read as a purely artificial restriction imposed for software convenience. It identifies the point at which an ambient persistence module can be represented by finite data without collapsing to a summary invariant that loses information. For the purposes of TAMER-OP, tameness is the hypothesis that makes it possible to keep the module, rather than discard it, and still work computationally.

3.3 Upsets, downsets, and fringe presentations

Finite encodings are one language for tameness. Another is given by indicator modules attached to order-theoretic regions.

Definition 16 (Upsets and downsets)**.** Let Q be a poset.

(i) An *upset* is a subset $U \subseteq Q$ such that $q \in U$ and $q \leq q'$ imply $q' \in U$.

(ii) A *downset* is a subset $D \subseteq Q$ such that $q' \in D$ and $q \leq q'$ imply $q \in D$.

For $q \in Q$, the corresponding principal upset and principal downset are

$$\uparrow q = \{q' \in Q : q \leq q'\}, \qquad \downarrow q = \{q' \in Q : q' \leq q\}.$$

Definition 17 (Indicator modules)**.** To an upset $U \subseteq Q$ one associates the *upset indicator module* $\Bbbk\{U\}$, defined by

$$\Bbbk\{U\}(q) = \begin{cases} \Bbbk, & q \in U, \\ 0, & q \notin U, \end{cases}$$

with the evident structure maps. Dually, a downset D determines a *downset indicator module* $\Bbbk\{D\}$.

On a finite poset, principal upset indicators play the role of indecomposable projective modules, and principal downset indicators play the role of indecomposable injective modules. This is why they are the correct basic pieces for exact algebra on finite posets.

Definition 18 (Finite fringe presentation)**.** A *finite fringe presentation* of a Q-module M consists of a morphism

$$\varphi : \bigoplus_{i=1}^{m} \Bbbk\{U_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{D_j\}$$

from a finite direct sum of upset indicator modules to a finite direct sum of downset indicator modules such that $M \cong \mathrm{im}(\varphi)$ [Mil20].

This presentation language is one of the key bridges between geometry and algebra in the library. Upsets encode where generators are born and persist upward in the order; downsets encode where relations become enforced; and the matrix of φ records how these pieces interact.

3.4 Why encoding is the right answer

The key fact is that tameness admits several equivalent finite descriptions.

Theorem 19 (Syzygy theorem, after Miller)**.** *For a module over a poset, the following finiteness conditions are equivalent: existence of a finite encoding, existence of a finite fringe presentation, and existence of finite resolutions by indicator modules [Mil20].*

This theorem explains why encoding is the right computational answer to the difficulties of multiparameter persistence. We describe three consequences of it now.

Firstly, it says that one does not need to choose between a geometric region picture, a presentation by births and deaths, and a homological description by resolutions. For tame modules, these are different finite views of the same object.

Secondly, it explains what is gained by encoding. We do not package a multiparameter module back into one clean barcode, because that is impossible in general. Instead, we pass to a finite poset on which the module is represented exactly by finite data. Once that finite representation has been constructed, one can carry out actual algebra: kernels, cokernels, change-of-poset operations, indicator resolutions, and derived functor calculations. Invariants such as rank functions, slice barcodes, Hilbert functions, Euler characteristic functions, and their associated signed measures are then extracted from this finite model, rather than substituted for it.

Thirdly, it explains why this approach does not lose information in the sense relevant to the library. The encoded module on the finite poset is not a secondary summary attached to the original module, but rather the finite algebraic object through which the original module is being represented. The purpose of the encoding step is therefore not compression for its own sake. It is to move from an ambient indexing space that is too large for direct exact computation to a finite model on which exact computation is possible.

This is the perspective adopted throughout the rest of the thesis. The next section therefore begins with the architecture of the encoding-centered system itself, because once the encoded finite-poset module has been built, the rest of the library follows from that choice.

4 Overview of TAMER-OP as an encoding-centered system

The previous section explained why finite encodings are mathematically legitimate and computationally necessary. This section explains how that idea informs the organization of TAMER-OP. We identify the stable mathematical interfaces that the implementation is built around; these are arranged so that many front-end objects feed a single encoding step, and that encoding step in turn feeds many downstream computations.

At the highest level, the library is organized around the composite workflow

$$\mathcal{I} \xrightarrow{\text{prepare}} \mathcal{P} \xrightarrow{\text{encode}} \mathcal{E} \xrightarrow{\text{analyze}} \mathcal{O},$$

where:

- $\mathcal{I}$ denotes raw inputs, such as data files, point clouds, graphs, images, or already structured presentation objects;
- $\mathcal{P}$ denotes pre-encoding mathematical objects, such as finite fringe modules, flanges over $\mathbb{Z}^n$, PL or box presentations over $\mathbb{R}^n$, and labeled cell complexes produced by ingestion;
- $\mathcal{E}$ denotes finite encodings, represented concretely by a finite poset together with an encoded module and the maps needed to interpret it back in the ambient parameter space; and
- $\mathcal{O}$ denotes downstream outputs, including categorical constructions, derived functors, invariants, feature vectors, serialized artifacts, and visual objects.

The crucial middle object $\mathcal{E}$ is what the codebase packages as an `EncodingResult`. Stripping away software metadata, the mathematical content of such an object is the tuple

$$(P, M, \pi, H).$$

Here P is a finite encoding poset, M is a finite-poset module on P, π is the classifier or encoding map from the ambient domain to P, and H is a pushed-down fringe presentation when such a presentation is available. Not every later calculation uses all four pieces, but together they constitute the canonical object available to the rest of the system.

This packaging is contextually important: the encoding step does not merely return a poset and forget how it arose; it returns a finite module, a mechanism for point location in the original parameter space, and enough provenance to support comparison, serialization, and visualization. From the user-facing point of view, this is the moment at which a raw front end object becomes a reusable algebraic object.

Figure 1 summarizes the resulting architecture.

The diagram is purposefully thin in the center but branched in input and output. The left side is diverse because the library is intended to accept front-end objects in the form most natural to their source. The middle is narrow because there is only one conceptual move that matters: build a finite encoding. The right side branches again because, once the encoded object exists, many different computational goals can be pursued from the same internal representation.

This thin diagrammatic section characterizing encoding identifies the precise place at which the computational problem changes style. Before encoding, one is still dealing with ambient parameter spaces, geometric representations, filtration choices, and presentation-specific subtleties. After encoding, one is dealing with a finite-poset module. The post-encoding world is therefore uniform even though the pre-encoding world is intentionally heterogeneous.

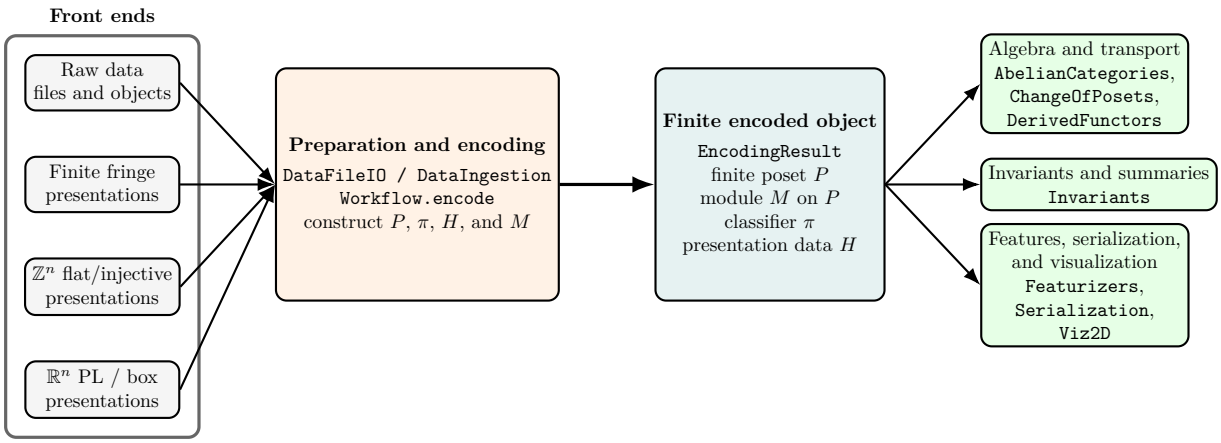

Figure 1: A system-level view of TAMER-OP. Multiple front ends are normalized into a single encoded object, and all later computation branches from that encoded object rather than from the original ambient representation.

This is also the main mathematical reason that TAMER-OP can do things that are awkward or impossible for existing libraries, which are generally built around architectures that prioritize only summaries and invariants. Once an actual encoded module has been retained, one may compute kernels, cokernels, equalizers, pushouts, projective and injective approximations, derived functors, and comparison transports before one ever decides which invariant or feature vector to extract. The encoded module is therefore not just a transient intermediate artifact, but the central object around which the rest of the pipeline is organized.

The rest of the thesis will follow the flow of figure 1, with the next chapter describing pre-encoding objects and data interfaces.

5 Pre-encoding objects and data interfaces

Sections 2 and 3 introduced the mathematical reason that a finite encoding should exist and should be the object on which exact computation is carried out. Before one can build such an encoding, however, one needs a finite front end description of the module to be encoded. This chapter describes the principal front ends accepted by TAMER-OP.

The guiding principle is as follows: the library is intentionally permissive about the form in which a module is first described, but intentionally strict about the object on which later algebra is done. On the input side, one may begin with a module already indexed by a finite poset, with a $\mathbb{Z}^n$-presentation adapted to finitely determined modules, with a piecewise-linear presentation over $\mathbb{R}^n$, or with raw topological data from which such a presentation can be constructed. On the output side of the encoding step there is only the working language of a module on a finite encoding poset.

5.1 Finite-poset presentations: finite fringe presentations and indicator data

The simplest front end occurs when the indexing poset is already finite. In that case no geometric compression is needed to obtain a finite combinatorial object, and the basic question is only how to package the module in a form that is easy to manipulate and easy to compare with later encoded outputs.

Let P be a finite poset. As in Section 3, an upset $U \subseteq P$ determines an indicator module $\Bbbk\{U\}$, and a downset $D \subseteq P$ determines an indicator module $\Bbbk\{D\}$. Since P is finite, these indicator modules are themselves finite diagrams of finite-dimensional vector spaces.

Definition 20 (Finite fringe presentation on a finite poset)**.** Let P be a finite poset and let $\Bbbk$ be a field. A *finite fringe presentation* on P consists of finitely many upsets $U_1, \dots, U_m \subseteq P$, finitely many downsets $D_1, \dots, D_r \subseteq P$, and a morphism

$$\varphi : \bigoplus_{i=1}^{m} \Bbbk\{U_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{D_j\},$$

with the associated module defined as

$$M := \operatorname{im}(\varphi).$$

Equivalently, once bases are chosen on the one-dimensional indicator summands, the presentation is specified by a matrix

$$\Phi \in \Bbbk^{r \times m}$$

subject to the support condition that $\Phi_{ji} = 0$ whenever $U_i \cap D_j = \varnothing$.

The matrix description is especially concrete on a finite poset. For each vertex $p \in P$, define the active column and row sets

$$I(p) = \{i \in \{1, \dots, m\} : p \in U_i\}, \qquad J(p) = \{j \in \{1, \dots, r\} : p \in D_j\}.$$

Evaluating the presentation at p yields a linear map

$$\varphi_p : \Bbbk^{I(p)} \longrightarrow \Bbbk^{J(p)}$$

whose matrix is the submatrix $\Phi_{J(p),I(p)}$. Thus the fiber at p is

$$M(p) = \operatorname{im}(\varphi_p), \qquad \dim_{\Bbbk} M(p) = \operatorname{rank}(\Phi_{J(p),I(p)}).$$

In other words, the module is read off from the same finite matrix together with the membership pattern of each vertex in the chosen upsets and downsets.

This front end matters for two central reasons. Firstly, it is the cleanest correctness model in the whole library. When the user already knows the finite indexing poset, there is no mystery with the encoding; all later kernels, cokernels, Hom, Ext, and invariant computations can be checked directly against this finite presentation. Secondly, finite fringe presentations are also the natural output of the encoding step, not just merely one among many input formats. The point of encoding is precisely to replace a module on a large poset by a finite-poset module or, equivalently, by a finite fringe presentation on a finite region poset. For that reason the finite fringe language is both a front end description and the simplest model of the finite encoding.

5.2 $\mathbb{Z}^n$ front end: flange presentations and finitely determined data

The next front end is adapted to modules over $\mathbb{Z}^n$ with the coordinatewise order. This is the natural discrete setting for many multifiltration constructions, and it is also the setting in which the connection with multigraded commutative algebra is most direct [CZ09]. A $\mathbb{Z}^n$-module is a functor from the poset category $\mathbb{Z}^n$ to vector spaces, equivalently a multigraded representation with structure maps in the positive coordinate directions.

A particularly important class is formed by finitely determined modules.

Definition 21 (Finitely determined $\mathbb{Z}^n$-module)**.** A $\mathbb{Z}^n$-module M is *finitely determined* if there exists $a \in \mathbb{Z}^n$ such that for every $b \geq a$ and every standard basis vector e_i, the structure map

$$M(b \leq b + e_i) : M(b) \longrightarrow M(b + e_i)$$

is an isomorphism.

This condition says that, sufficiently far in the positive orthant, the module stops changing in any essential way. It is the discrete analogue of eventual constancy on regions, and it is one of the standard hypotheses under which finite descriptions become possible.

Miller's theory packages these modules using *flats* and *injectives*. Fix a subset $\tau \subseteq \{1, \dots, n\}$, thought of as a set of free coordinate directions, and a lattice point $b \in \mathbb{Z}^n$. The corresponding flat and injective regions are

$$F_{\tau,b} = \{a \in \mathbb{Z}^n : a_i \geq b_i \text{ for every } i \notin \tau\},$$

$$E_{\tau,b} = \{a \in \mathbb{Z}^n : a_i \leq b_i \text{ for every } i \notin \tau\}.$$

The coordinates in τ are unrestricted. Thus principal upsets and principal downsets occur as the special case $\tau = \varnothing$, while more general choices of τ allow one to model affine coordinate flats in the sense used in Miller's $\mathbb{Z}^n$-theory [Mil20].

Definition 22 (Flange presentation)**.** A *flange presentation* over $\mathbb{Z}^n$ consists of finitely many flats $F_1, \dots, F_m$, finitely many injectives $E_1, \dots, E_r$, and a matrix

$$\Phi \in \Bbbk^{r \times m}$$

with the support condition $\Phi_{ji} = 0$ whenever $F_i \cap E_j = \varnothing$. It presents the module

$$M := \operatorname{im}\left(\bigoplus_{i=1}^{m} \Bbbk\{F_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{E_j\} \right).$$

The local behavior is parallel to the finite fringe case. For a lattice point $g \in \mathbb{Z}^n$, let

$$I(g) = \{i : g \in F_i\}, \qquad J(g) = \{j : g \in E_j\}.$$

Then the fiber $M(g)$ is the image of the submatrix $\Phi_{J(g),I(g)}$, so fiber dimensions and local maps are determined by the active flats and injectives at g. What changes, compared to the finite-poset case, is that the ambient poset is infinite. The flange presentation therefore serves as a compact input language whose membership predicates can be evaluated anywhere in $\mathbb{Z}^n$, but which is intended to be encoded before more substantial algebra is done.

5.3 $\mathbb{R}^n$ front-end: PL fringes and box presentations

For modules over $\mathbb{R}^n$, one needs a geometric version of the same idea. The relevant inputs are piecewise-linear upsets and downsets, represented by finite polyhedral data. This is the correct front end for modules whose behavior changes along finitely many polyhedral boundaries, and it is also the natural setting for geometric constructions arising from stratified topology and sheaf-theoretic viewpoints on persistence [KS18, Mil23].

Definition 23 (PL upset and PL downset)**.** A *PL upset* in $\mathbb{R}^n$ is an upward-closed subset that is a finite union of convex polyhedra. A *PL downset* is defined dually as a downward-closed finite union of convex polyhedra.

The front end presentation is then the direct real-parameter analogue of a fringe presentation.

Definition 24 (PL fringe presentation)**.** A *PL fringe presentation* over $\mathbb{R}^n$ consists of finitely many PL upsets $U_1, \dots, U_m \subseteq \mathbb{R}^n$, finitely many PL downsets $D_1, \dots, D_r \subseteq \mathbb{R}^n$, and a matrix

$$\Phi \in \Bbbk^{r \times m}$$

with $\Phi_{ji} = 0$ whenever $U_i \cap D_j = \varnothing$. The associated module is again the image of the induced morphism

$$\bigoplus_{i=1}^{m} \Bbbk\{U_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{D_j\}.$$

At a point $x \in \mathbb{R}^n$, one evaluates the same active-set construction as before,

$$I(x) = \{i : x \in U_i\}, \qquad J(x) = \{j : x \in D_j\},$$

and the fiber is computed from the submatrix $\Phi_{J(x),I(x)}$. Thus the formal algebra is unchanged, with change happening in the geometry of membership and intersection. In the real setting, a correct implementation must be able to decide whether a point lies in one of the finitely many prescribed polyhedral regions and must be able to identify the induced constant regions of the presentation.

An important specialization is the axis-aligned box backend. Here one restricts to upsets and downsets of the form

$$U_\ell = \{x \in \mathbb{R}^n : x_i \geq \ell_i \text{ for all } i\}, \qquad D_u = \{x \in \mathbb{R}^n : x_i \leq u_i \text{ for all } i\}.$$

This format covers a large and practically relevant class of examples. Additionally, and importantly, they admit a much more direct encoding procedure, covered in the next section. So, ultimately, the polyhedral front end is mathematically more general, while the box front end is algorithmically much simpler, and the library supports both.

5.4 Raw data, filtrations, and labeled cell complexes

Not every user begins with a presentation by upsets and downsets. In topological data analysis the more common starting point is raw data together with a rule for assigning parameter values to cells, simplices, edges, or vertices. For this reason TAMER-OP includes a data-ingestion layer whose role is to convert raw data into one of the presentation objects discussed above, usually by way of a labeled cell complex.

Definition 25 (Labeled cell complex)**.** Let K be a finite cell complex. A *labeled cell complex* over a poset Q is the data of a grade

$$\deg(\sigma) \in Q$$

for each cell σ of K, subject to the monotonicity condition that if τ is a face of σ, then

$$\deg(\tau) \leq \deg(\sigma).$$

Equivalently, the chain groups of K become Q-graded and the boundary maps are degree-nonincreasing in the usual persistence sense.

From such a labeled cell complex one obtains persistence modules by taking homology or cohomology degreewise. In the one-parameter setting this is the standard construction underlying persistent homology [ZC05]; in the multiparameter setting the same principle applies, but the grade now lies in $\mathbb{Z}^n$, $\mathbb{R}^n$, or another suitable poset.

The data-ingestion layer accepts several kinds of raw or partially structured input. The most basic data objects are point clouds, graphs, and images. In addition, the user may begin with an already constructed labeled cell complex, a multicritical labeled cell complex in which a cell carries several incomparable minimal grades, or a compressed simplex-tree representation of a multifiltered simplicial complex. The point is not that all of these objects are mathematically identical, but that each of them can be functorially transformed into a labeled cell complex, from which an encoding can then be built.

The transformation is controlled by a typed filtration specification. For point clouds, representative families include Vietoris–Rips, density-modified Rips, landmark Rips, Delaunay-type, alpha-type, and related constructions. For graphs, the library supports lower-star, clique lower-star, edge-weighted, geodesic, centrality-based, and other graph-specific filtrations. For images and cubical data, lower-star and distance-based filtrations play the analogous role. The output of

this stage is still pre-encoding, being a finite graded object with an explicitly recorded parameter assignment, not yet a finite encoding poset.

The next stage of the thesis is to explain how all these front ends we have discussed are converted into a finite encoding poset and a finite module on that poset.

6 The encoding step

The previous chapter described the mathematical objects that can appear before encoding. We now turn to the central algorithmic problem of the thesis: given one of those front-end descriptions, how does one construct a finite poset, a finite module on that poset, and a classifier that remembers how the original ambient parameter space maps into the finite model? This is the central step around which the rest of TAMER-OP is organized.

6.1 Statement of the encoding problem

Let Q be a poset, often $\mathbb{Z}^n$, $\mathbb{R}^n$, or a finite grid or region poset derived from data. Suppose one is given a Q-module M indirectly, by a front-end presentation or by raw filtered data. The encoding problem is to construct finite data

$$(P, M_P, \pi, H)$$

(whose codebase object is the `EncodingResult`) with the following properties:

(i) P is a finite poset;

(ii) $\pi\colon Q \to P$ is a monotone classifier map;

(iii) M_P is a module on P with finite-dimensional fibers;

(iv) when a presentation by indicator modules is available, H records a pushed down fringe presentation on P; and

(v) the original module is recovered, up to the relevant notion of equality, by pullback along π.

At the theoretical level, this is precisely the notion of finite encoding introduced in section 3. The finite-poset module M_P is the object used by later categorical and homological routines; the classifier π is the mechanism by which one returns to the ambient parameter space.

Additionally, the encoded object should be reusable. In particular, later stages of the library require:

(a) repeated point location in the ambient parameter space,

(b) efficient comparison of multiple modules on a common finite model,

(c) compatibility with module-level algebra on finite posets,

(d) and access to enough geometric information to support weighted summaries, feature extraction, and visualization.

6.2 Signatures, constant regions, and finite model posets

The unifying idea behind all of the encoding algorithms is the same. One starts with a finite family of generators and relations, usually expressed as upsets and downsets in the ambient parameter poset. A point of the parameter space is then classified by which generators are active and which relations are active there. Points with the same activity pattern belong to the same constant region of the module. The finite encoding poset is the poset of such regions.

To make this precise, suppose first that a module is presented by a morphism

$$\varphi\colon \bigoplus_{i=1}^{m} \Bbbk\{U_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{D_j\}$$

of indicator modules on a poset Q, where the U_i are upsets and the D_j are downsets. For each parameter value $q \in Q$ define two binary membership vectors

$$y(q) = (y_1(q), \dots, y_m(q)) \in \{0,1\}^m, \qquad z(q) = (z_1(q), \dots, z_r(q)) \in \{0,1\}^r$$

by

$$y_i(q) = 1 \iff q \in U_i, \qquad z_j(q) = 1 \iff q \notin D_j.$$

The use of the complement in the second formula is deliberate. The sets

$$\{q \in Q : y_i(q) = 1\} = U_i, \qquad \{q \in Q : z_j(q) = 1\} = Q \setminus D_j$$

are both upsets, so both parts of the signature vary monotonically with q.

Definition 26. Define the *signature* of q to be the pair

$$\sigma(q) = (y(q), z(q)).$$

Two points are placed in the same constant region when they have the same signature.

Proposition 27 (Why signatures determine the encoded module)**.** *Let $M = \mathrm{im}(\varphi)$ be presented as above. If $q, q' \in Q$ satisfy $\sigma(q) = \sigma(q')$, then the fibers $M(q)$ and $M(q')$ are canonically identified. More generally, if $q \le q_1$ and $q' \le q'_1$ with*

$$\sigma(q) = \sigma(q') \qquad \text{and} \qquad \sigma(q_1) = \sigma(q'_1),$$

then the structure maps

$$M(q \le q_1), \qquad M(q' \le q'_1)$$

are represented by the same linear map.

Proof. At any $q \in Q$, the active generator and relation sets are

$$I(q) = \{i : q \in U_i\}, \qquad J(q) = \{j : q \in D_j\}.$$

These are determined by the signature, since $I(q)$ is read from $y(q)$ and $J(q)$ is read from the complement of $z(q)$. The presentation evaluated at q is therefore the linear map

$$\varphi_q\colon \Bbbk^{I(q)} \longrightarrow \Bbbk^{J(q)}$$

with matrix the submatrix $\Phi_{J(q),I(q)}$ of the global presentation matrix. If $\sigma(q) = \sigma(q')$, then $I(q) = I(q')$ and $J(q) = J(q')$, so φ_q and $\varphi_{q'}$ are the same linear map. Hence their images are canonically the same vector space, giving the identification $M(q) \cong M(q')$.

For the second claim, the structure map from q to q_1 is induced by the evident inclusions of active generator sets and active relation sets between the two signatures. If the source and target signatures are the same in the two comparable pairs, those inclusions are the same, so the induced linear map is the same. □

Proposition 27 is the algebraic reason that signatures suffice. It says that once source and target signatures are known, the local presentation matrix and the corresponding structure map are known as well. Thus the module factors through the poset of realized signatures.

The partial order on signatures is the natural one:

$$(y, z) \leq (y', z') \qquad \Longleftrightarrow \qquad y_i \leq y'_i \text{ for all } i, \quad z_j \leq z'_j \text{ for all } j.$$

Because both parts of the signature are monotone with respect to the ambient order on Q, the classifier map

$$\pi \colon Q \longrightarrow P, \qquad q \longmapsto \sigma(q)$$

is monotone when P is the poset of realized signatures ordered componentwise. In the discrete back end this poset is implemented as a structured signature poset; in denser or more generic situations one may materialize the same order relation as an ordinary finite poset.

Figure 2 summarizes the conceptual pipeline. The main observation is that the same pattern appears in every front end: one first induces finitely many cells or regions in the ambient parameter space, then one labels those cells by signatures, merges cells with the same signature, and finally orders the resulting regions by monotone reachability.

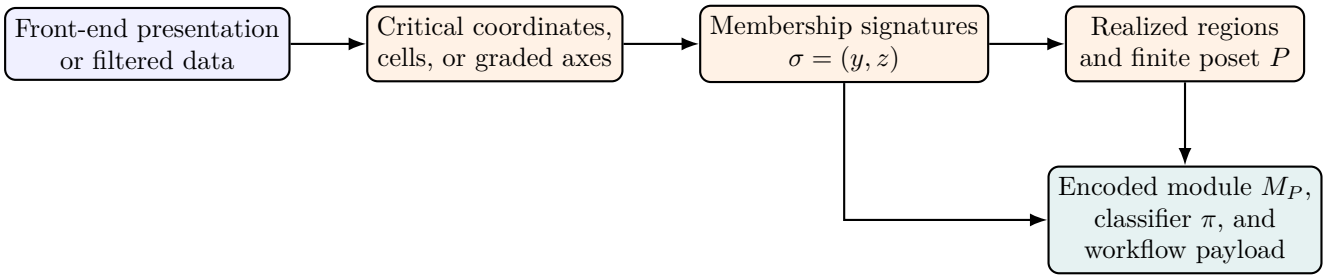

Figure 2: The common structure of the encoding algorithms. The front end determines finitely many critical cells or grades; these are labeled by membership signatures; equal signatures are merged into regions; and the resulting finite region poset supports the encoded module and classifier map.

6.3 The finite-poset case

The easiest encoding case is the one in which the input already lives on a finite poset. If a module is already given as a finite-poset module or through a finite fringe presentation on a finite poset P, then there is no geometric compression left to perform. One may simply take

$$\pi = \mathrm{id}_P, \qquad H = M,$$

and regard the given module as already encoded.

This seemingly trivial case is nevertheless architecturally important, as it fixes the target language for every other encoding routine. The point of the nontrivial encoders is not to invent some new sort of output object, but to reduce back to this finite case. This is why the user-facing workflow still packages a finite poset input into the same kind of result object used everywhere else, which is the `EncodingResult` object.

In practice, this finite case also provides the normalization point for correctness checks. Whenever a more geometric encoder is tested, the resulting finite module is compared against the module obtained by evaluating the original front end presentation on the encoded regions. Thus, the finite poset case is conceptually degenerate but mathematically definitive.

6.4 Encoding flange presentations over $\mathbb{Z}^n$

We now come to the first actually nontrivial encoding algorithm. Suppose the input is a flange presentation over $\mathbb{Z}^n$, as introduced in section 5. Concretely, we are given finitely many flats

$F_1, \ldots, F_m$, finitely many injectives $E_1, \ldots, E_r$, and a matrix

$$\Phi \in \Bbbk^{r \times m}$$

with the usual support condition. The ambient poset $\mathbb{Z}^n$ is infinite, so the purpose of the encoder is to replace these data by a finite region poset.

The first observation is that the membership of a lattice point in a flat or injective can change only when one crosses one of finitely many coordinate thresholds. Indeed, each flat and each injective is defined by finitely many affine coordinate inequalities, so along each coordinate axis there are only finitely many critical integers coming from the generator boundaries. Collect these critical integers on axis i into a sorted list

$$c_{i,1} < c_{i,2} < \cdots < c_{i,k_i}.$$

These values cut the lattice into rectangular slab cells. A slab cell records, for each coordinate, whether the point lies before the first critical integer, exactly on a critical integer, or between consecutive critical integers. Any two lattice points lying in the same slab cell have the same membership in every flat and every injective. Hence the signature is constant on each slab cell.

The implementation reflects this observation directly. A `ZnEncodingMap` stores:

- the ambient dimension n;
- the critical-coordinate lists on each axis;
- the realized y- and z-signature rows;
- a representative lattice point for each encoded region;
- a dictionary from signatures to region indices;
- and, when available, a direct slab-cell-to-region lookup table.

The algorithm proceeds in four conceptual steps.

Step 1: build the critical coordinate axes. From the flange generators one extracts the finitely many relevant coordinate thresholds. These are stored axis by axis and determine the discrete slab decomposition. The point of this step is not yet to decide which regions are nonempty or distinct, but only to identify a finite search space in which the signature can change.

Step 2: evaluate realized signatures on slab representatives. Each slab cell admits a representative lattice point, and the code evaluates that representative against the flat and injective families. The resulting signature is the pair

$$\sigma(g) = (y(g), z(g)), \qquad y_i(g) = 1 \iff g \in F_i, \qquad z_j(g) = 1 \iff g \notin E_j.$$

Cells with the same signature are merged. In the implementation the signature rows are bit-packed, so that large families of generators can still be compared quickly and stored compactly.

Step 3: form the finite region poset. The realized signatures are partially ordered by componentwise inclusion. This yields the finite signature poset

$$P = \{\text{realized signatures}\}, \qquad \sigma \leq \sigma' \iff \sigma \text{ is componentwise contained in } \sigma'.$$

This is the finite encoding poset. The classifier map

$$\pi : \mathbb{Z}^n \to P$$

sends a lattice point to the signature of the slab cell containing it.

Step 4: push the presentation to the finite poset. The original flats and injectives are pushed forward to upsets and downsets in P. The presentation matrix is then monomialized: entries that correspond to pairs of pushed-forward generator and relation regions that do not intersect are set to zero. This enforces the same support condition on the encoded finite fringe presentation that the original flange satisfied in the lattice.

The output of this stage is therefore not just the finite poset P but an actual finite fringe module on P, together with the classifier map and region representatives. The workflow layer then materializes, when needed, the corresponding finite poset module.

There are also two important implementation consequences.

Firstly, point location becomes fast. Once the slab-cell lookup table has been built, locating a lattice point means determining which slab cell it lies in and then reading off the region index. This is one of the reasons the $\mathbb{Z}^n$ encoding route supports later invariant queries efficiently.

Secondly, encodings can be reused. The code records an encoding profile determined by the ambient dimension and the underlying flat and injective families. This allows later pushforward plans, region masks, and compiled lookup data to be cached and reused across repeated queries or across collections of modules that share the same geometric encoding problem but differ only in coefficients or in the presentation matrix. This is especially important when several modules are encoded at once over the same poset.

Indeed, this simultaneous encoding case is built into the workflow. When a finite collection of flanges is encoded together, the library first constructs a single common region poset P and a single classifier map π, and only then pushes each flange to that common target. This is essential for later comparison and bifunctorial constructions such as Hom and Ext, since those operations require their arguments to live on a common finite poset model.

6.5 Encoding PL and box presentations over $\mathbb{R}^n$

The situation with real parameters is formally similar but geometrically richer. A PL fringe presentation again consists of finitely many upsets, finitely many downsets, and a matrix relating them. The encoding problem is to partition the relevant part of $\mathbb{R}^n$ into finitely many regions on which the signature is constant and then to pass to the corresponding finite poset.

In TAMER-OP this is implemented through two complementary back ends.

The box-specialized backend

When the input consists of axis-aligned box upsets and box downsets, the encoding can be performed much more directly than in the general polyhedral case. Along each axis one again extracts the finitely many coordinate values occurring as lower or upper bounds of the generators. These values define a rectangular cell decomposition of the ambient space. Every open or half-open cell in this decomposition has constant membership in every box generator, so signatures are constant on cells.

The implementation exploits this aggressively. Rather than re-evaluating every generator independently on every cell representative, it traverses the cells in boustrophedon order and updates the signature incrementally. Crossing a coordinate boundary can only affect the generators whose threshold is met at that boundary, so the signature update is sparse. In this way, the full grid of cells is traversed once, realized signatures are deduplicated, and a direct cell-to-region lookup table is built.

This backend therefore has a very transparent geometry:

(i) collect coordinate thresholds;

(ii) build the rectangular cell grid;

(iii) traverse cells and compute signatures;

(iv) merge equal signatures into regions;

(v) order the regions by componentwise signature inclusion;

(vi) push the original presentation to the resulting finite poset.

The output classifier is a `PLEncodingMapBoxes`. Besides the region signatures and representatives, it stores the coordinate axes, the mixed-radix cell shape and strides, and a cell-to-region table. As in the lattice case, this supports fast repeated point location. When a point lies away from a cell boundary, location is a direct mixed-radix lookup. Near a boundary, the implementation can either use a fast convention or a more careful verified mode, depending on verification requirements.

The general PL/polyhedral backend

Axis-aligned boxes occur often enough that they merit their own specialized backend, but they are not the only natural real parameter front end. In many situations the relevant generators and cogenerators are fully polyhedral: they are cut out by arbitrary linear inequalities and therefore meet along oblique facets rather than along coordinate hyperplanes. The ambient parameter space is then divided by a finite collection of polyhedral boundaries, and the resulting constant regions of the module need no longer be rectangular or even combinatorially simple. The general PL backend is designed for exactly this richer setting.

The first implementation decision is to represent each convex piece by an explicit halfspace description

$$Ax \leq b$$

stored over the exact coefficient field $\mathbb{Q}$. This is important because the encoder needs an exact description of the geometry while it is deciding which signatures are actually realized, and floating-point approximations may be too inexact. Additionally, we make the design choice where point membership tests should not depend on a particular version of an external polyhedral package in `Julia` like `Polyhedra` or `CDDLib`. So, the basic polyhedral object stores the H-representation itself, and routine membership in a convex piece or a finite union of such pieces is evaluated directly from the stored inequalities.

This leads to the second design decision, which is to avoid building the full polyhedral arrangement explicitly. In principle one could intersect every boundary hyperplane with every other and attempt to construct the full cell decomposition of the arrangement. In practice that would be the wrong primitive: the arrangement remembers far more geometry than the module itself uses, and its combinatorial complexity can be much larger than the number of distinct module regions. The backend therefore prioritizes signatures rather than arrangements.

Suppose the input presentation is

$$\varphi\colon \bigoplus_{i=1}^{m} \Bbbk\{U_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{D_j\},$$

where each U_i is a PL upset and each D_j is a PL downset, each given as a finite union of convex polyhedra. As in the earlier discussion of signatures, one seeks the pairs

$$(y, z) \in \{0,1\}^m \times \{0,1\}^r$$

that are actually realized by points of $\mathbb{R}^n$, where y_i records membership in U_i and z_j records non-membership in D_j. The naive search space is still finite, but potentially large, so the

implementation is organized around a branch-and-feasibility strategy that only keeps signatures that are geometrically realizable.

The central observation is that membership in a finite union is a disjunction over its convex pieces, while non-membership is the conjunction of being outside every piece. The encoder resolves those two logical forms differently.

If a signature requires a point to lie inside a PL upset or downset, the algorithm chooses one convex piece from that union and adds its full H-representation to the running system. If a signature requires a point to lie outside a PL upset or downset, the algorithm does not try to complement the whole polyhedron directly. Instead, for each convex piece it chooses one facet inequality to violate. Thus "outside a union" becomes a finite family of exact feasibility branches, each one encoded by selecting, for every relevant piece, a single facet that must be crossed. Internally, these outside conditions are implemented as strict violations

$$a^\top x > b$$

which are converted into exact closed inequalities

$$a^\top x \geq b + \varepsilon$$

for a fixed rational slack $\varepsilon > 0$. In this way, strictness is handled symbolically rather than numerically. Once the chosen "inside" and "outside" conditions have been assembled, the backend builds a single exact polyhedron from the combined inequalities and asks whether it is empty. If it is nonempty, the signature is realized; if it is empty, the branch is discarded.

This procedure is encapsulated in the feasibility enumeration routine. Its output is not the whole arrangement, but a list of realized signatures together with a polyhedral representative for the corresponding region and, when available, a witness point extracted from a V-representation. The witness is not logically necessary for correctness, but it is extremely useful later: it gives the backend a cheap representative point for region-level queries, candidate ordering in point location, and later geometric summaries. The enumeration routine also collapses repeated signatures. Distinct feasibility branches can describe different polyhedral pieces of the same signature class, but from the point of view of the encoded module those pieces are equivalent. It is therefore the signature, not the raw region geometry, that determines the finite encoding poset.

Once the realized signatures have been collected, the backend constructs the finite region poset. There are two closely related choices. In the first, the realized signatures themselves are ordered componentwise, producing the signature poset. In the second, one passes to a denser "uptight" refinement derived from the same signatures. The user-facing choice is recorded through the option `poset_kind`. In either case the result is a finite poset P whose vertices correspond to module regions rather than to geometric cells of the ambient arrangement.

The original PL presentation is then pushed down to P. For each original upset generator U_i, one forms its image on P by marking precisely those realized signatures in which the ith y-bit is active; dually, for each downset generator D_j, one marks those realized signatures in which the jth z-bit is inactive. These masks are closed upward or downward inside the finite poset to obtain honest upsets and downsets on P. The coefficient matrix is then monomialized: if the pushed-forward upset and downset corresponding to an entry Φ_{ji} are disjoint in the finite region poset, that entry is set to zero.

The output of the backend is therefore a triple

$$(P, H, \pi),$$

where P is the finite region poset, H is the pushed-down finite fringe module on P, and π is a classifier from $\mathbb{R}^n$ to P. The classifier is represented by a `PLEncodingMap`, which stores not only

the signature data and region representatives, but also a compiled collection of fast geometric surrogates used for repeated point queries.

This compiled point-location layer is one of the main practical devices that makes the backend usable. Exact polyhedral feasibility is too expensive to rerun every time an invariant routine asks where a point lies. The encoder therefore precomputes floating-point copies of the halfspace systems for each region, in both strict and slightly relaxed forms. These do not replace the exact geometry used during encoding, but they provide a fast and robust numerical filter for later queries. Given a point x, the classifier tests candidate regions by evaluating the stored float halfspaces with a controlled tolerance. Because the regions are already known to be exact from the encoding stage, this floating-point layer is a search mechanism rather than a source of mathematical ambiguity.

The candidate set itself is reduced by several prefilters. The simplest uses witness points: regions are partially ordered according to one witness coordinate, and a query need only inspect a short window around the corresponding coordinate of the query point. When the ambient dimension is at least three and the witness cloud has informative variance in several coordinates, the backend builds a multi-projection prefilter that keeps a small candidate family consistent across several projected orderings at once. In dimensions one and two, when enough regions are present and exact region bounding boxes can be obtained, the backend also builds a coarse spatial bucket index from region bounding boxes. This allows a query point to be routed immediately to a small list of nearby candidate regions. The net effect is that point location is usually much cheaper than a full scan of all polyhedral regions, even though the exact encoded geometry may be complicated.

The same philosophy appears in the batched query routines. If a large matrix of query points is supplied, the backend groups points by bucket and applies the same candidate sets repeatedly, with threading heuristics that switch on only when the expected work is large enough. In other words, the PL backend is not only exact at construction time; it is also engineered for repeated evaluation after construction. That matters because many later invariants, especially slice-based and geometric ones, call the classifier many times.

The general PL backend is also the route by which the library supports *common encoding* of several PL fringe presentations at once. In that case the upsets and downsets from all inputs are pooled, feasibility enumeration is performed once on the combined family, and the resulting common region poset is then sliced back into one pushed-down fringe presentation for each input module. This is a substantial advantage of the general backend over the box-specialized route: once several modules have been encoded into the same finite poset, later comparisons such as Hom, Ext, and invariant-level distances can be carried out directly on a common finite model.

Finally, the PL backend is also where region geometry begins to enter the picture. Because the encoding map stores actual polyhedral regions, the library can later ask geometric questions about those regions inside a finite window: their volume, bounding box, centroid, boundary measure, adjacency pattern, or principal directions. To make repeated geometry queries tractable, the backend provides a dedicated `PolyInBoxCache`. This cache stores, lazily and relative to a fixed ambient box, the intersections of regions with that box, their exact or approximate V- and H-representations, active-region masks, axis-aligned bounding boxes, facet data, and exact weights or centroids when these have already been computed. Thus the geometric layer is not an afterthought added after encoding. It is a natural extension of the same compiled region structure and will later serve as the prelude to weighted invariants, feature extraction, and visualization.

For all of these reasons, the general PL backend should be viewed as more than merely "the slow but general one." Its role is to make finite encoding over $\mathbb{R}^n$ mathematically correct even when the front-end geometry is genuinely polyhedral, while still remaining tractable through a

careful separation of concerns: exact arithmetic for region construction, lightweight halfspace tests for repeated queries, signature-first enumeration rather than full arrangement construction, and cache-driven reuse for point location and region geometry.

6.6 Geometry attached to an encoding map

The encoding step produces more than just a finite poset and a classifier. It also produces a finite stratification of the ambient parameter space. Once this stratification exists, one may ask geometric questions about its regions.

At the most basic level, every encoding provides region representatives and, when possible, axes derived from the encoding. In grid-like settings these axes are simply the critical coordinates together with representative values for the intervening slabs. More refined questions are backend-sensitive. For a region one may ask for a bounding box, coordinate widths, a centroid, a diameter, or a measure of its boundary. For a whole encoding one may ask which regions are adjacent, how large the interfaces between them are, or how much volume each region occupies inside a chosen window of the ambient parameter space.

This geometric layer should be understood correctly. It is not another pre-encoding input type. The regions whose geometry is being measured are not the original front-end generators, but the strata produced by the encoding itself. In other words, region geometry is an attribute of the encoded decomposition of parameter space, not of the initial presentation data.

That distinction matters because the later invariant layer uses exactly this sort of information. Weighted Hilbert summaries, support measures, adjacency-based summaries, shape descriptors, and region-based visualizations all depend on geometric data attached to the encoding map. The present chapter therefore introduces the idea, while the later chapter on invariants will use it systematically.

From the implementation point of view, the division is again backend-sensitive. For lattice encodings and box-based real encodings, many geometric quantities can be read off directly from the critical-coordinate lists and the region lookup tables. For more general PL encodings, the same questions require a richer geometric interface. The codebase isolates those geometric queries in a separate layer so that invariant and visualization routines can ask for region weights, boundary size, or adjacency without having to know which backend supplied them.

6.7 From filtered data to finite encodings

The front ends discussed so far begin with explicit presentations. The data-ingestion layer begins earlier, with filtered data. Its task is to convert raw data into a finite algebraic object that can be encoded by the same workflow.

There are two main internal representations at this stage: labeled cell complexes and packed simplex trees.

A labeled cell complex records cells, boundary matrices, and the multigrade of each cell. This is the correct representation for a general finite filtered cell complex. A `SimplexTreeMulti` records the same sort of information for simplicial data in a more compressed form. Concretely, it stores the vertices of all simplices in flattened arrays together with offset tables, simplex dimensions, and the grade data attached to each simplex. The point of this packed representation is that one can traverse simplices and grades without first expanding the full boundary structure into a dense or redundant intermediate object.

The user-facing contract is the routine

$$\texttt{build_graded_complex(data, filtration)} \rightsquigarrow (G, \text{axes, orientation}).$$

The filtration family, for example a Rips-type, lower-star, graph-based, cubical, or Delaunay-type filtration, assigns grades to the combinatorial pieces of the raw dataset. The result is either a labeled cell complex directly or, when the route is simplicial and compactly representable, a simplex-tree object together with the same essential grading information.

The encoding step for such objects differs from the fringe and flange cases in one important respect. Here the front-end object is already finite. The issue is not that the indexing poset is infinite, but that the labeled cell complex is not yet organized as a reusable finite-poset module. The ingestion layer therefore turns the finite set of occurring grades into axes, forms the corresponding grid or product-of-chains poset, and then computes the appropriate cochain or cohomology object on that finite poset.

More concretely, the ingestion workflow performs the following steps.

(i) Starting from the raw data and filtration specification, build a labeled cell complex or a simplex tree.

(ii) Extract the finite coordinate axes determined by the grades. Optional quantization, axis policies, and coarsening may be applied at this stage.

(iii) Build the finite grid poset P determined by those axes and by the chosen orientation convention.

(iv) Construct a cochain complex over P, either lazily from the packed simplex tree or directly from the labeled cell complex.

(v) Compute the requested output stage: a cochain complex, a module, a fringe presentation, a flange (in the integer-axis case), cohomology dimensions, or a full `EncodingResult`.

The presence of the simplex-tree route is especially important. It means that the pipeline does not need to materialize every intermediate presentation eagerly in order to reach an encoded module. When the input is simplicial, the packed simplex tree can be kept long enough for the boundary and cohomology computations to be built lazily. This is one of the main reasons the ingestion layer scales better than a naive expand-everything approach.

At the same time, the endpoint is the same as for the presentation-based encoders. Once the cohomology module on the finite grid poset has been built, it is packaged as an encoded object with a classifier map. Thus filtered data reaches the same finite-poset world as flanges and PL fringes do, only by a different route.

This is also the right place to clarify what it means to encode *from a graded module*. If the user already has a finite labeled cell complex or a module supported on finitely many grades, then the set of grades itself determines finite axes. The encoding problem is then not one of discovering signatures from infinite geometry, but of building the finite grid poset that makes those grades reusable as a module object and equips them with a classifier map and workflow metadata. In this sense, data-derived modules and presentation-derived modules meet at the same conceptual endpoint: a finite encoded module on a finite poset.

6.8 Common encodings, compiled evaluation, and reusable results

Many later computations involve more than one module. Hom, Ext, interleaving-style comparisons, common feature extraction, and benchmarking against shared inputs all require the objects being compared to be represented against a common finite combinatorial model whenever possible. For this reason the encoding layer supports common encodings.

In the lattice case, several flange presentations can be encoded simultaneously by constructing a single common signature poset and a single classifier map, then pushing each presentation to that common target. In the real case, common encoding is supported for collections of compatible PL inputs, with some backend-specific restrictions. If modules are already finite-poset modules on

different finite posets, the library also supports comparison by passing to a common refinement, typically a product poset.

The other side of reuse is repeated evaluation. Once an encoding has been built, later invariant and visualization routines may need to locate thousands or millions of ambient parameter values. A purely formal encoding map would be enough for correctness, but it would not be enough for performance. The library therefore compiles encoding maps into wrappers that cache axes, representatives, and hot-path metadata, and that provide repeated evaluation routines such as batched point location.

This is why the workflow distinguishes between an abstract encoding map and a compiled encoding. The abstract object records the mathematics; the compiled object records the mathematics together with the cached data needed to make repeated evaluation cheap. The same distinction appears again in the result layer. An `EncodingResult` is not just a tuple of mathematical objects, but a reusable workflow object designed to survive later algebra, invariant computation, serialization, and benchmarking.

6.9 Complexity, diagnostics, and the role of the encoding chapter

A fully formal complexity theory for the entire encoding layer would be long and not especially illuminating, because the dominant costs depend strongly on the front end. Nevertheless, the main cost centers are easy to identify.

For flange presentations over $\mathbb{Z}^n$, the dominant tasks are extracting critical coordinates, traversing slab cells, deduplicating realized signatures, building the finite signature poset, and pushing the presentation to that poset. For box presentations over $\mathbb{R}^n$, the story is similar, except that the ambient cells are geometric rather than purely discrete. For general PL presentations, region construction and point location are more expensive because the cell structure is no longer rectangular. For data ingestion, the main additional costs are filtration construction, boundary assembly or traversal, and cohomology computation before the encoded module is even packaged.

The most important practical failure mode is region explosion. If too many critical thresholds are present, the induced cell structure can become too large to be useful. The library therefore exposes construction budgets, axis coarsening policies, and backend-selection diagnostics. Other diagnostics concern degenerate boundaries, unsupported backend combinations, and stage-specific restrictions, such as the requirement that flange output from data ingestion only makes sense when the final axes are integral.

These diagnostics are not peripheral. They are part of the value of making the encoding step explicit. A library that treats encoded modules as its central internal object must also explain when such an object can be built cheaply, when the geometry is too large, and which backend assumptions are being used. The encoding layer is therefore both the conceptual heart of the library and the place where its computational promises are either realized or fail.

From this point onward, the thesis works almost entirely inside the category of modules over a finite encoding poset.

7 Bibliography notes

References

[Mil20] E. Miller. *Homological algebra of modules over posets.* arXiv:2008.00063, 2020.

[Waa24] L. Waas. *Notes on abelianity of categories of finitely encodable persistence modules.* arXiv:2407.08666, 2024.

[Mil23] E. Miller. *Stratifications of real vector spaces from constructible sheaves with conical microsupport*. *Journal of Applied and Computational Topology*, 7(3):473–489, 2023. doi:10.1007/s41468-023-00112-1.

[KS18] M. Kashiwara and P. Schapira. *Persistent homology and microlocal sheaf theory*. *Journal of Applied and Computational Topology*, 2(1–2):83–113, 2018. arXiv:1705.00955.

[CB15] W. Crawley-Boevey. *Decomposition of pointwise finite-dimensional persistence modules*. *Journal of Algebra and Its Applications*, 14(5):1550066, 2015. doi:10.1142/S0219498815500668.

[ZC05] A. Zomorodian and G. Carlsson. *Computing persistent homology*. *Discrete & Computational Geometry*, 33:249–274, 2005. doi:10.1007/s00454-004-1146-y.

[CZ09] G. Carlsson and A. Zomorodian. *The theory of multidimensional persistence*. *Discrete & Computational Geometry*, 42:71–93, 2009. doi:10.1007/s00454-009-9176-0.

[CEH07] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. *Stability of persistence diagrams*. *Discrete & Computational Geometry*, 37(1):103–120, 2007. doi:10.1007/s00454-006-1276-5.

[ELZ02] H. Edelsbrunner, D. Letscher, and A. Zomorodian. *Topological persistence and simplification*. *Discrete & Computational Geometry*, 28:511–533, 2002. doi:10.1007/s00454-002-2885-2.

[Les15] M. Lesnick. *The theory of the interleaving distance on multidimensional persistence modules*. *Foundations of Computational Mathematics*, 15(3):613–650, 2015. doi:10.1007/s10208-015-9255-y.

[LW15] M. Lesnick and M. Wright. *Interactive visualization of 2-D persistence modules*. arXiv:1512.00180, 2015.

[RIVET] M. Lesnick and M. Wright. *RIVET: The rank invariant visualization and exploration tool* (software). `https://github.com/rivetTDA/rivet`.

[LS24] D. Loiseaux and H. Schreiber. *multipers: Multiparameter persistence for machine learning*. *Journal of Open Source Software*, 9(103):6773, 2024. doi:10.21105/joss.06773.

[CDSGO16] F. Chazal, V. de Silva, M. Glisse, and S. Y. Oudot. *The Structure and Stability of Persistence Modules*. SpringerBriefs in Mathematics. Springer, 2016. doi:10.1007/978-3-319-42545-0.

[Oud15] S. Y. Oudot. *Persistence Theory: From Quiver Representations to Data Analysis*. Mathematical Surveys and Monographs, Vol. 209. American Mathematical Society, 2015.

[EH10] H. Edelsbrunner and J. L. Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.

[Mac98] S. Mac Lane. *Categories for the Working Mathematician*. Graduate Texts in Mathematics, Vol. 5. Springer, 2nd edition, 1998. doi:10.1007/978-1-4757-4721-8.

[Wei94] C. A. Weibel. *An Introduction to Homological Algebra*. Cambridge Studies in Advanced Mathematics, Vol. 38. Cambridge University Press, 1994.

[Eis95] D. Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry*. Graduate Texts in Mathematics, Vol. 150. Springer, 1995. doi:10.1007/978-1-4612-5350-1.

[MS05] E. Miller and B. Sturmfels. *Combinatorial Commutative Algebra*. Graduate Texts in Mathematics, Vol. 227. Springer, 2005. doi:10.1007/b138602.